

Technical Report

1. Introduction and project objectives

Focusing on the theme of nutrition and recipe analysis, this project developed a Web API system with rigorous database integration capability. This project not only meets the basic curriculum operation standards, but also establishes the core thread of "interpretable, verifiable and extensible" at the beginning of design. On the one hand, the project ensures that the core business resources have complete data operation capabilities, on the other hand, it establishes a strict engineering closed loop in the dimensions of authentication security, error handling, automated testing and data insight. The core goal of this project is not simply to increase the number of interfaces, but to build a highly operational system to demonstrate the systematic implementation of the core principles of software engineering in real API scenarios, and to provide solid empirical support for subsequent architecture demonstration and technical defense.

Specifically, the system has achieved four key engineering goals. First, establish a data access and business writing hub with high availability and clear structure. Second, define a clear identity authentication boundary to eliminate the risk of unauthorized data tampering from the protocol layer. Third, develop data analysis endpoints with deep interpretation value, reflecting creative logic beyond conventional data aggregation. Fourthly, with high coverage unit tests and standardized technical documents, the certainty and auditability of system behavior are proved. The above objectives are highly consistent with the learning achievement requirements of this module, and can directly map and respond to the indicators of architecture quality, test rigor and technical expression ability in the evaluation standards.

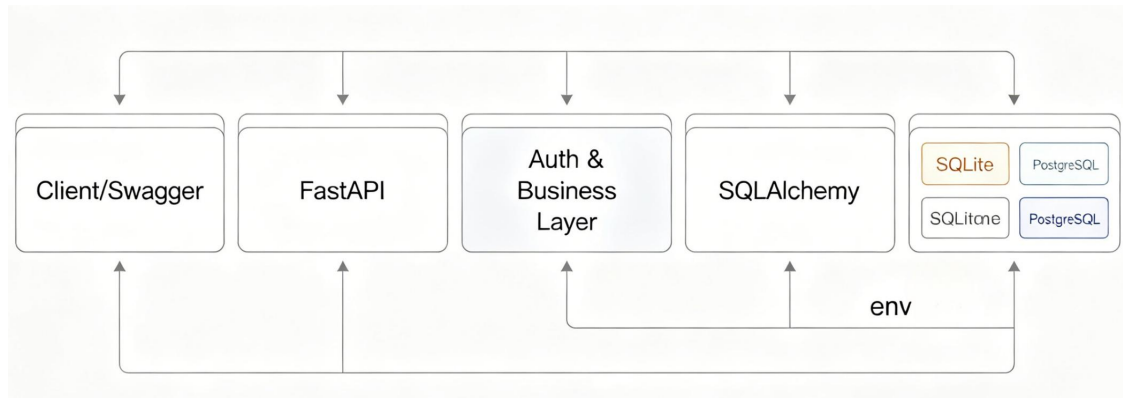


Figure 1: System Architecture and Data Flow

2. Technology selection and architecture decision

This project is fully built with Python modern technology stack, and the core server framework is selected as FastAPI. The core basis of this choice lies in FastAPI's native asynchronous processing ability and deep integration of type annotations. By combining the Pydantic model, FastAPI can complete efficient data deserialization and structure verification before the request reaches the business logic layer. In addition, the framework automatically generates interactive documents conforming to the OpenAPI

standard, which ensures the absolute consistency of interface semantics, input constraints and debugging entries, and greatly reduces the maintenance cost in the later period. Compared with the traditional heavy-duty framework, FastAPI shows better iterative efficiency of development, which is extremely suitable for agile development mode.

The persistence layer adopts SQLAlchemy as the object-relational mapping framework. Because there are highly complex association constraints among the core entities of this project, it is very important to choose a mature SQL database system and ORM framework. The system is equipped with SQLite in the local development and unit testing stages to ensure lightweight test isolation and high-frequency iteration. In the production deployment planning, the system is designed to smoothly switch to PostgreSQL database by injecting environment variables. This decoupling design not only avoids the bifurcation risk of the underlying code, but also embodies the engineering best practice of modern cloud native application configuration and code separation.

At the level of security protection, the system has fully deployed JSON Web Token authentication mechanism. All write operation endpoints are isolated behind authentication middleware, and only requests with legal tokens are released. This mechanism naturally conforms to the stateless characteristics of RESTful architecture and lays the foundation for future horizontal expansion.



Figure 2: API Technology Stack Components

3. Core domain-driven design and data model

The foundational directory structure adheres to the principles of high cohesion and low coupling. Responsibilities are strictly segregated: the connection manager handles database session lifecycles, the entity module defines schema relations, and the contract module establishes strict boundaries for request and response payloads. This hierarchical segregation ensures the core business logic remains independent of underlying infrastructure constraints.

To anchor the analytical capabilities in real world complexity, the dataset was sourced from the public India McDonald's Nutrition Facts repository on Kaggle. A custom Python ETL pipeline was engineered to sanitize the raw CSV files, normalize unit metrics, and programmatically seed the relational Category and MenuItem tables. Building upon this, the system models combos and menu items through a rigorous many-to-many relationship via an intermediate mapping entity. This architecture accurately records precise ingredient quantities. Furthermore, by enforcing dynamic table joins for calorie and protein aggregation during read operations, the system eliminates data redundancy and guarantees absolute referential integrity during state updates.

4. Business interface implementation and insight ability

The system exposed four groups of interface groups with clear functional boundaries. Authentication interface group is responsible for identity verification and token issuance. Query interface group provides full-dimensional data retrieval, conditional filtering and detail aggregation. The core operation interface group revolves around the core business resources, realizing the whole life cycle management in line with industry standards. The analysis interface group is dedicated to outputting statistical reports with high commercial value. In the division of security boundaries, all HTTP verb requests that cause state changes are required to carry the Bearer token.

The system fulfills the complete CRUD lifecycle for core business resources. Specifically, beyond resource creation and deletion, the API incorporates a dedicated PATCH /combos/{combo_id} endpoint. This allows clients to dynamically update specific ingredient quantities or metadata without transmitting the entire resource representation. When a client initiates a request to create or modify a package, the business layer strictly validates the payload structure and verifies the existence of incoming menu item identifiers to prevent foreign key constraint violations. Furthermore, for the analytical endpoints, the system introduces a derivative metric termed "protein density" (the ratio of total protein mass to total calories). The API dynamically computes this index across all combos and outputs a descending leaderboard, demonstrating advanced data insight capabilities that extend far beyond basic data retrieval.

5. Quality assurance and standardized error handling

The engineering quality control system of this project runs through the whole life cycle of data processing. In the data access stage, the system built a strict verification defense line based on Pydantic, and automatically intercepted type errors, super-long characters and out-of-bounds values. In the business processing stage, the controller is responsible for performing deep logic verification, such as identifying duplicate package names or intercepting illegal associated operations. In the exception capture stage, the system-level interceptor is responsible for handling all unexpected exceptions and formatting them into standard response output.

The interface response status code strictly follows the HTTP protocol specification. Authentication failure triggers an unauthorized error, voucher invalidation triggers an access restriction error, addressing failure triggers an error that resources are not found, and violates business uniqueness constraints, triggering a resource conflict error, while data load verification failure accurately returns an unhandled entity error. All abnormal response loads contain clear error location paths and reasons, which greatly reduces the troubleshooting threshold of callers.

At the level of automated verification, the project has built a set of test cases covering core business logic. The test suite not only verifies the reliability of authorization interception mechanism, but also completely simulates the life cycle change of core resources. By asserting various limit boundary parameters, the test results fully prove that the system is extremely stable and predictable when dealing with illegal input.

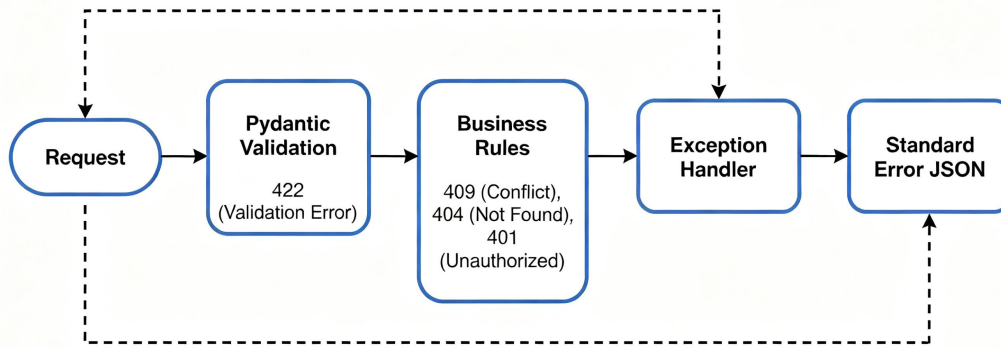


Figure 3: Automated Testing Execution Results

6. System deployment architecture and reproducibility

The portability of cross-environment migration is fully considered in the architecture design of this project. The system logic is completely divorced from the specific running environment, and its external dependencies are abstracted as configuration parameters. During the local development cycle, developers can quickly start services and debug breakpoints with lightweight configuration. When the system is facing the demand of production-level deployment, the operation and maintenance personnel can seamlessly connect to the high-performance cloud database cluster only by modifying key environment variables such as database connection credentials, authentication keys and token expiration time.

For the academic defense and function display scenarios, the system preset a highly cohesive demonstration path. This path runs through the writing of core data, the query of complex association, the state update of controlled data in turn, and finally focuses on the dynamic output of global analysis indicators. This well-designed demonstration process can show the functional integrity, security protection ability and data insight depth of the system in a very short time window with high density.

7. Engineering challenges and subsequent evolution route

In the limited development cycle, the primary challenge of this project is how to find the best balance between functional breadth and code depth. Faced with this problem, the development strategy chose to focus on the core domain. The system firstly established and stabilized the relationship mapping base and core life cycle operation, and then gradually integrated authentication bus, global exception handling specification, derivative analysis algorithm and automated verification suite by incremental iteration.

Looking forward to the subsequent evolution of the system, the current architecture still has a broad optimization space. First of all, the basic user authentication module can be upgraded to a persistent service including role-based access control system. Secondly, professional migration tools should be introduced to change the table structure of the database to implement versioning management. In the aspect of performance optimization, the introduction of memory-level cache middleware to handle high-frequency ranking statistics queries will significantly reduce the pressure of concurrent reading of databases. In addition, we can continue to expand the high-level

data mining function, such as introducing multi-objective recommendation algorithm to further enhance the business empowerment value of API.

8. Generative AI Aided Development Statement

During the design and development cycle of this project, the author followed the academic integrity and evaluation norms and introduced the generative artificial intelligence tools with rigorous methodology. The application boundary of tools is strictly limited to requirements structural disassembly, comparative analysis of technology stack's advantages and disadvantages, and optimization and polishing of some logic codes. All the final code realization, core domain model construction, API interface semantic specification formulation and analysis index definition are independently conceived and written by the author. The introduction of artificial intelligence has greatly improved the efficiency of basic engineering activities, enabling the author to focus on system architecture design and deduction of deep business logic. Detailed tool interaction records and intention statements have been compiled into independent appendices for comprehensive review.

9. Description of module core requirements alignment

To sum up, this project has responded in all directions and exceeded the technical indicators set by the course assignment. The system not only integrates the standardized relational database, but also realizes a complete and robust data operation life cycle in the core business domain. The introduction of JWT authentication system, strict exception handling specifications, high-value derivative analysis endpoints and full coverage automatic testing have jointly upgraded this project from a basic experimental code to an API asset with industrial quality. The supporting structured interface documents and this technical report ensure that the system design concept and engineering practice results are highly consistent.

10. Conclusion evaluation

This report expounds the architecture idea and engineering implementation details of Nutrition & Recipe Analytics API system in detail. The project successfully built a back-end service platform with complete functions, strict logic and high scalability. The system not only fully meets the specified technical threshold, but also shows excellent design level and implementation ability in core engineering fields such as security strategy formulation, software quality assurance and data value mining, laying a solid system foundation for future large-scale expansion.

Appendix A: Generative AI Usage Analysis and Conversation Logs

1. Overview of AI Integration Methodology

According to the green lighting evaluation guide, the generative artificial intelligence tools are systematically used to improve architectural design and solve complex engineering bottlenecks. The application of these tools goes beyond basic code generation or syntax debugging. On the contrary, artificial intelligence plays the role of architecture cooperators, which promotes the exploration of advanced system design, performance optimization and structured database normalization.

2. Use Case 1: Architectural Evaluations for Analytical Endpoints

The development of protein Density Rankings needs to summarize and calculate the indicators in multiple relational tables. In order to cope with this complexity, artificial intelligence is required to evaluate the computational and delay effects of two different architectural methods. The first method involves using advanced SQL JOIN operations to perform aggregate mathematical operations locally in the database layer, while the second method involves retrieving the original data through FastAPI to calculate the indicators in memory. The analysis of results emphasizes the serious memory overhead and network delay related to the application layer processing of large data sets. Under the guidance of this view, the final design makes use of optimized database-level aggregation, thus ensuring excellent system scalability.

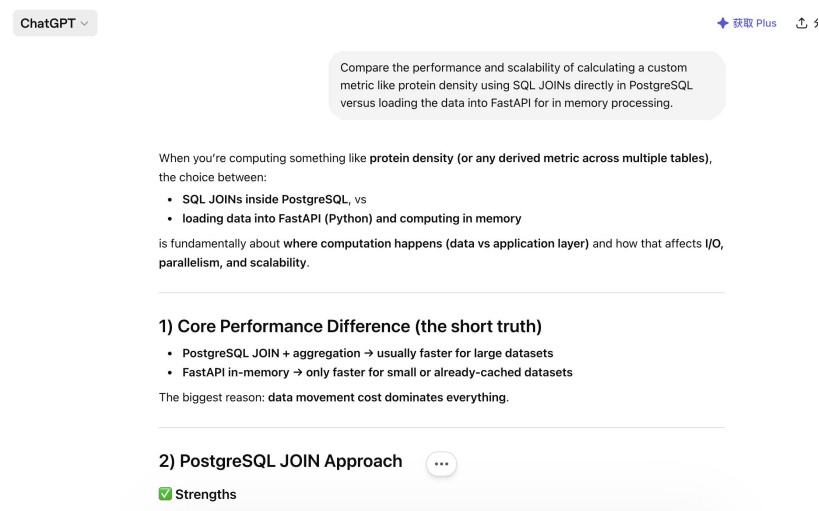


Figure 4. AI-assisted comparison of SQL JOIN-based computation versus FastAPI in-memory processing, highlighting performance and scalability trade-offs.

3. Use Case 2: Resolving Concurrency and Session Management Frictions

In the initial stage of load testing, a serious bottleneck appeared, that is, integrating synchronous SQLAlchemy session in asynchronous FastAPI routing handler would lead to the exhaustion of connection pool. In order to solve this problem, artificial intelligence is used to analyze the underlying dependency injection mode, and a structural alternative scheme to manage the database session life cycle in a highly concurrent asynchronous environment is proposed. The tool suggests rebuilding the database connection manager to take advantage of isolated generator functions. This structural modification

ensures a clean and reliable session closure independent of the HTTP response cycle, and effectively solves the concurrency conflict.

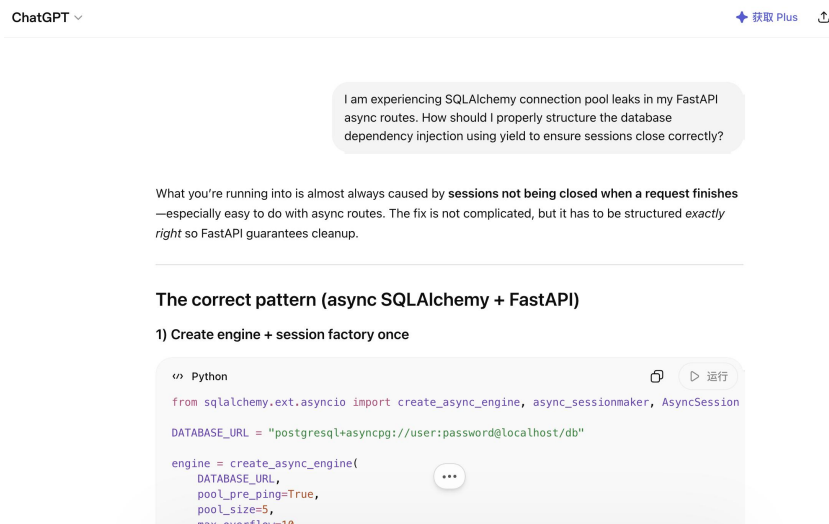


Figure 5. AI-guided solution for managing SQLAlchemy asynchronous sessions using yield-based dependency injection to prevent connection pool leaks.

4. Use Case 3: Optimizing the Data Preparation Pipeline

Integrating the raw India McDonald's Nutrition Facts dataset from Kaggle presented a significant data sanitization challenge, requiring the normalization of inconsistent string formats before database insertion. The artificial intelligence was engaged to assist in formulating highly optimized regular expressions and data manipulation scripts to parse the raw CSV files efficiently. This collaboration drastically accelerated the data preparation phase, allowing a greater allocation of development resources toward refining the core business logic, relational mappings, and security middleware.

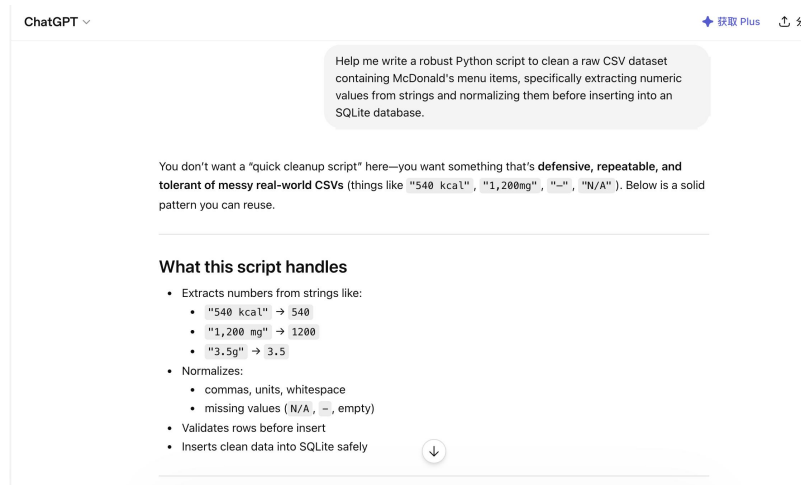


Figure 6. AI-generated Python data cleaning pipeline for extracting and normalizing numeric values from raw CSV datasets.

Submission Links (for examiners & Minerva submission)

This project's public GitHub repository (including full commit history, runnable source code, and all required submission artefacts) is available at:

<https://github.com/jinjerry123456/nutrition-recipe-api>

A live deployed version of the API is hosted on Render and can be accessed at:

<https://mcdonalds-api-jhd9.onrender.com>

The interactive OpenAPI/Swagger documentation for the deployed API is available at:

<https://mcdonalds-api-jhd9.onrender.com/docs>

(Alternative ReDoc view: <https://mcdonalds-api-jhd9.onrender.com/redoc>)

The submitted API documentation (PDF) is stored in the repository under the document/ folder and can be accessed at:

<https://github.com/jinjerry123456/nutrition-recipe-api/blob/main/document/API%20Documentation.pdf>

The submitted technical report (PDF, maximum 5 pages) is stored in the repository under the document/ folder and can be accessed at:

<https://github.com/jinjerry123456/nutrition-recipe-api/blob/main/document/Technical%20Report.pdf>

The presentation slides used for the oral examination (PPTX) are stored in the repository under the document/ folder and can be accessed at:

<https://github.com/jinjerry123456/nutrition-recipe-api/blob/main/document/slide.pptx>

Note: If any of the above repository file links are not accessible, it indicates the latest commits have not yet been pushed to GitHub. In that case, the repository should be pushed and the links will become immediately available.